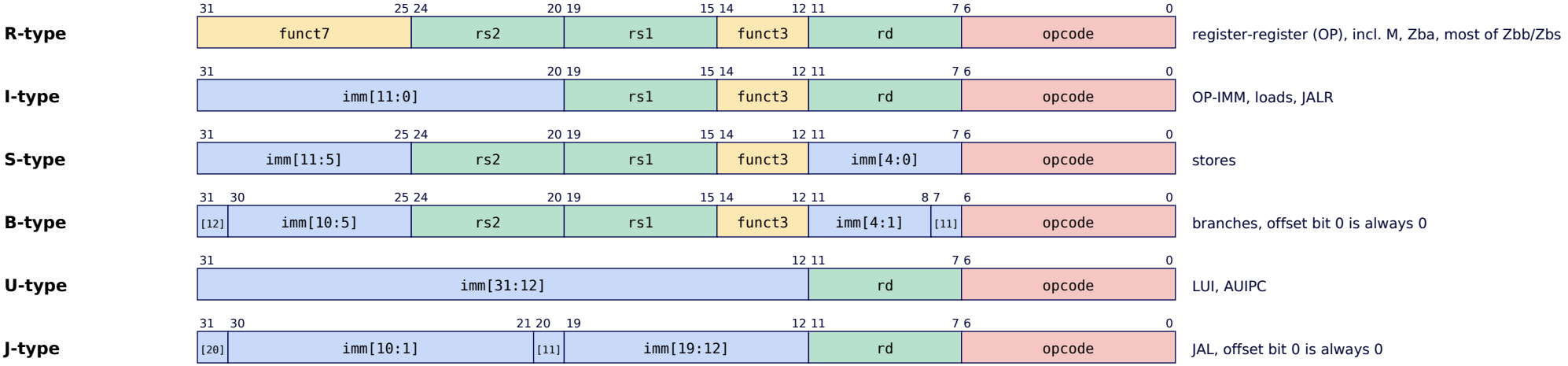
RV32IMB instruction cheat sheet

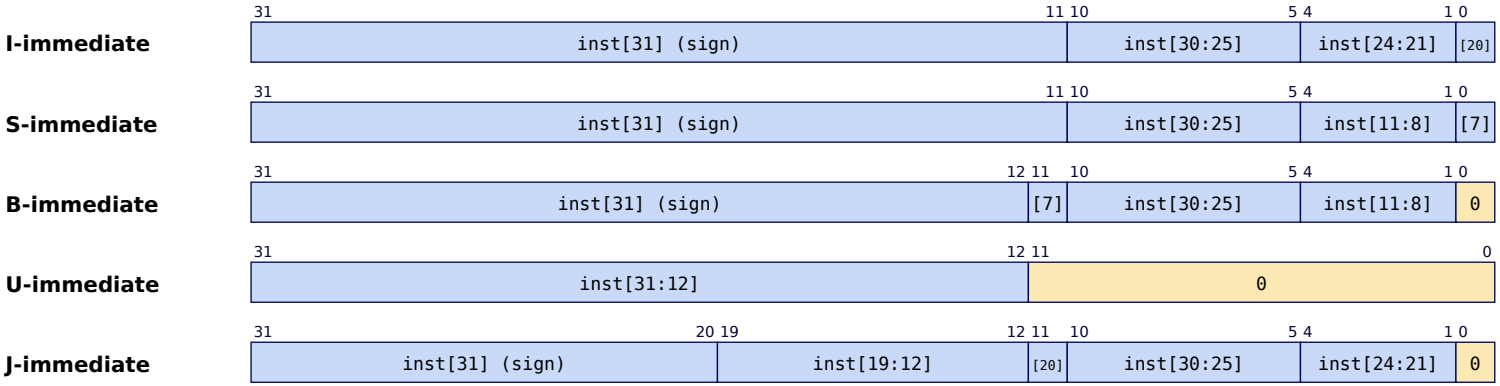
RV32I base + M (multiply/divide) + B (= Zba + Zbb + Zbs bit manipulation) + Zicsr (CSR access) + the privileged instructions mret, wfi, sret, sfence.vma. All instructions are 32 bits. **sext** = sign-extend, **zext** = zero-extend, all registers are 32 bits. Immediates in examples are decimal unless written 0x…; branch and jump offsets are byte offsets relative to the instruction itself. The **signed int32** column is the machine word read as a two's-complement number (what a Factorio signal carries).

Field colours: opcode | funct3 / funct7 / fixed | register | immediate

Base instruction formats (Sec. 2.2 / 2.3)

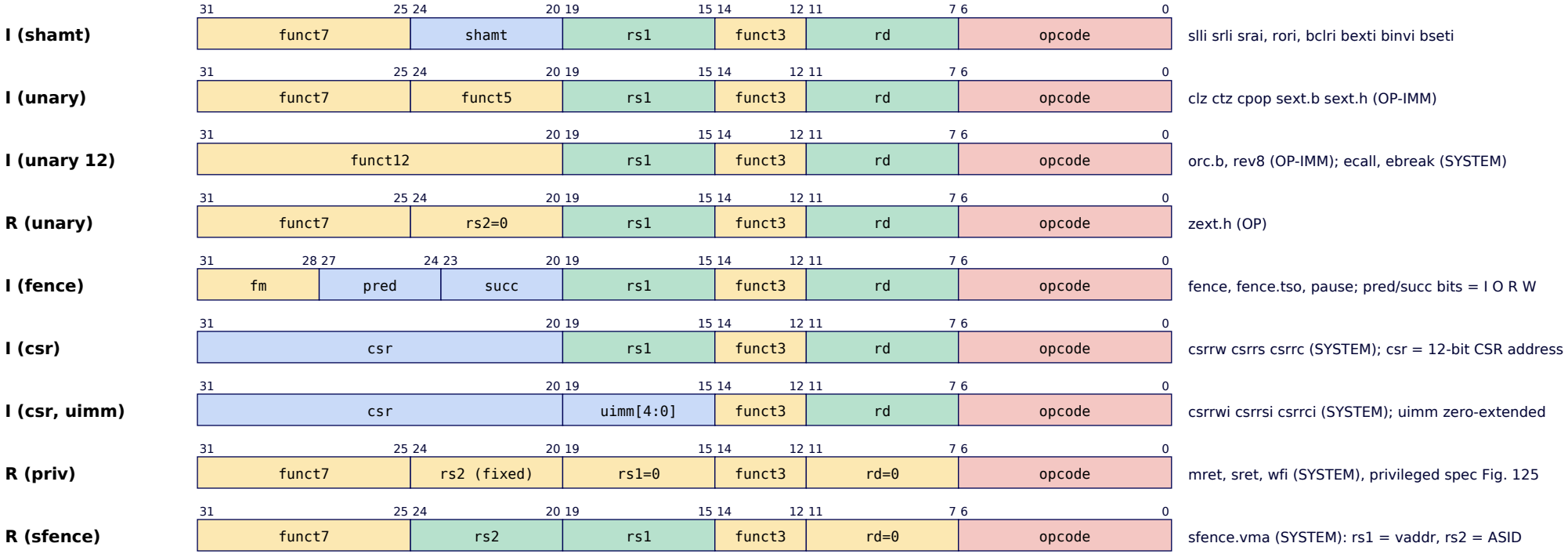


Immediate values produced by each format (Figure 1): bits of the 32-bit immediate, labelled with the instruction bits that fill them



The sign bit of every immediate is always inst[31]. Ranges: I/S −2048…2047 · B −4096…4094 (even) · J −1048576…1048574 (even) · U: imm20 0…0xFFFFF (value = imm20 × 4096) · shamt 0…31.

Format variants used by specific instructions



Major opcodes used (Table 72, inst[1:0] = 11)

Opcode	inst[6:0]	Hex	Dec	Formats	Instructions
LOAD	**0000011**	0x03	3	I	lb lh lw lbu lhu
MISC-MEM	**0001111**	0x0F	15	I (fence)	fence fence.tso pause
OP-IMM	**0010011**	0x13	19	I, I (shamt), I (unary)	addi slti sltiu xori ori andi slli srli srai clz ctz cpop sext.b sext.h rori orc.b rev8 bclri bexti binvi bseti
AUIPC	**0010111**	0x17	23	U	auipc
STORE	**0100011**	0x23	35	S	sb sh sw
OP	**0110011**	0x33	51	R, R (unary)	add sub sll slt sltu xor srl sra or and mul mulh mulhsu mulhu div divu rem remu sh1add sh2add sh3add andn orn xnor max maxu min minu zext.h rol ror bclr bext binv bset
LUI	**0110111**	0x37	55	U	lui
BRANCH	**1100011**	0x63	99	B	beq bne blt bge bltu bgeu
JALR	**1100111**	0x67	103	I	jalr
JAL	**1101111**	0x6F	111	J	jal
SYSTEM	**1110011**	0x73	115	I (csr), I (csr, uimm), I (system), R (priv), R (sfence)	ecall ebreak csrrw csrrs csrrc csrrwi csrrsi csrrci mret wfi sret sfence.vma

Registers (x0 is hard-wired to 0; ABI names accepted by the tools)

Reg	ABI	Reg	ABI	Reg	ABI	Reg	ABI
x0	zero	**x8**	s0 / fp	**x16**	a6	**x24**	s8
x1	ra	**x9**	s1	**x17**	a7	**x25**	s9
x2	sp	**x10**	a0	**x18**	s2	**x26**	s10
x3	gp	**x11**	a1	**x19**	s3	**x27**	s11
x4	tp	**x12**	a2	**x20**	s4	**x28**	t3
x5	t0	**x13**	a3	**x21**	s5	**x29**	t4
x6	t1	**x14**	a4	**x22**	s6	**x30**	t5
x7	t2	**x15**	a5	**x23**	s7	**x31**	t6

CSR names accepted by the tools (any 12-bit number 0…0xFFF also works)

Addr	Name	Addr	Name	Addr	Name	Addr	Name
0x300	**mstatus**	0x320	**mcountinhibit**	0xB80	**mcycleh**	0xF11	**mvendorid**
0x301	**misa**	0x340	**mscratch**	0xB82	**minstreth**	0xF12	**marchid**
0x302	**medeleg**	0x341	**mepc**	0xC00	**cycle**	0xF13	**mimpid**
0x303	**mideleg**	0x342	**mcause**	0xC01	**time**	0xF14	**mhartid**
0x304	**mie**	0x343	**mtval**	0xC02	**instret**	0xF15	**mconfigptr**
0x305	**mtvec**	0x344	**mip**	0xC80	**cycleh**		
0x306	**mcounteren**	0xB00	**mcycle**	0xC81	**timeh**		
0x310	**mstatush**	0xB02	**minstret**	0xC82	**instreth**		

Addresses from riscv-privileged.pdf Sec. 2.2: Table 4 (unprivileged counters, read-only) and Table 7 (machine-level CSRs). The *h variants are the upper 32 bits on RV32.

Instructions

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
RV32I: integer register-immediate								
addi	rd, rs1, imm	I	OP-IMM 0010011	000	–	rd = rs1 + sext(imm). Overflow is ignored.	addi x1, x2, 23	0x01710093 **24182931**
slti	rd, rs1, imm	I	OP-IMM 0010011	010	–	rd = (rs1 < sext(imm)) ? 1 : 0, signed compare	slti x5, x6, -1	0xFFF32293 **-843117**
sltiu	rd, rs1, imm	I	OP-IMM 0010011	011	–	rd = (rs1 < sext(imm)) ? 1 : 0, unsigned compare (imm is sign-extended first, then treated as unsigned)	sltiu x5, x6, 1	0x00133293 **1258131**
xori	rd, rs1, imm	I	OP-IMM 0010011	100	–	rd = rs1 ^ sext(imm)	xori x5, x6, -1	0xFFF34293 **-834925**
ori	rd, rs1, imm	I	OP-IMM 0010011	110	–	rd = rs1 | sext(imm)	ori x5, x6, 0x0F0	0x0F036293 **251880083**
andi	rd, rs1, imm	I	OP-IMM 0010011	111	–	rd = rs1 & sext(imm)	andi x5, x6, 255	0x0FF37293 **267612819**
slli	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	001	0000000	rd = rs1 << shamt (logical left shift, shamt 0..31)	slli x5, x6, 4	0x00431293 **4395667**
srli	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	101	0000000	rd = rs1 >> shamt, logical (zeros shifted in)	srli x5, x6, 4	0x00435293 **4412051**
srai	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	101	0100000	rd = rs1 >> shamt, arithmetic (sign bit copied in)	srai x5, x6, 4	0x40435293 **1078153875**

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
RV32I: integer register-register								
add	rd, rs1, rs2	R	OP 0110011	000	0000000	rd = rs1 + rs2. Overflow is ignored.	add x3, x4, x5	0x005201B3 **5374387**
sub	rd, rs1, rs2	R	OP 0110011	000	0100000	rd = rs1 − rs2	sub x3, x4, x5	0x405201B3 **1079116211**
sll	rd, rs1, rs2	R	OP 0110011	001	0000000	rd = rs1 << rs2[4:0] (logical left shift)	sll x3, x4, x5	0x005211B3 **5378483**
slt	rd, rs1, rs2	R	OP 0110011	010	0000000	rd = (rs1 < rs2) ? 1 : 0, signed	slt x3, x4, x5	0x005221B3 **5382579**
sltu	rd, rs1, rs2	R	OP 0110011	011	0000000	rd = (rs1 < rs2) ? 1 : 0, unsigned	sltu x3, x4, x5	0x005231B3 **5386675**
xor	rd, rs1, rs2	R	OP 0110011	100	0000000	rd = rs1 ^ rs2	xor x3, x4, x5	0x005241B3 **5390771**
srl	rd, rs1, rs2	R	OP 0110011	101	0000000	rd = rs1 >> rs2[4:0], logical	srl x3, x4, x5	0x005251B3 **5394867**
sra	rd, rs1, rs2	R	OP 0110011	101	0100000	rd = rs1 >> rs2[4:0], arithmetic	sra x3, x4, x5	0x405251B3 **1079136691**
or	rd, rs1, rs2	R	OP 0110011	110	0000000	rd = rs1 | rs2	or x3, x4, x5	0x005261B3 **5398963**
and	rd, rs1, rs2	R	OP 0110011	111	0000000	rd = rs1 & rs2	and x3, x4, x5	0x005271B3 **5403059**

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
RV32I: upper immediates								
lui	rd, imm20	U	LUI 0110111	–	–	rd = imm20 << 12 (lower 12 bits zero). Builds 32-bit constants (imm20: 0..0xFFFFF).	lui x5, 0x12345	0x123452B7 **305418935**
auipc	rd, imm20	U	AUIPC 0010111	–	–	rd = pc + (imm20 << 12). Builds pc-relative addresses.	auipc x5, 0x10	0x00010297 **66199**

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
RV32I: control transfer (offsets in bytes, relative to this instruction)								
jal	rd, offset	J	JAL 1101111	–	–	rd = pc + 4; pc = pc + offset. Offset is even, ±1 MiB.	jal ra, 2048	0x001000EF **1048815**
jalr	rd, offset(rs1)	I	JALR 1100111	000	–	t = pc + 4; pc = (rs1 + sext(offset)) & ~1; rd = t	jalr x1, 12(x5)	0x00C280E7 **12746983**
beq	rs1, rs2, offset	B	BRANCH 1100011	000	–	if rs1 == rs2: pc = pc + offset. Offset is even, ±4 KiB.	beq x1, x2, -8	0xFE208CE3 **-31421213**
bne	rs1, rs2, offset	B	BRANCH 1100011	001	–	if rs1 ≠ rs2: pc = pc + offset	bne x1, x2, 16	0x00209863 **2136163**
blt	rs1, rs2, offset	B	BRANCH 1100011	100	–	if rs1 < rs2 (signed): pc = pc + offset	blt x1, x2, -8	0xFE20CCE3 **-31404829**
bge	rs1, rs2, offset	B	BRANCH 1100011	101	–	if rs1 ≥ rs2 (signed): pc = pc + offset	bge x1, x2, 8	0x0020D463 **2151523**
bltu	rs1, rs2, offset	B	BRANCH 1100011	110	–	if rs1 < rs2 (unsigned): pc = pc + offset	bltu x1, x2, -4096	0x8020E063 **-2145329053**
bgeu	rs1, rs2, offset	B	BRANCH 1100011	111	–	if rs1 ≥ rs2 (unsigned): pc = pc + offset	bgeu x1, x2, 4094	0x7E20FFE3 **2116091875**

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
RV32I: loads and stores (address = rs1 + sext(offset))								
lb	rd, offset(rs1)	I	LOAD 0000011	000	–	rd = sext(mem8[addr]), 8-bit value sign-extended	lb x5, 0(x6)	0x00030283 **197251**
lh	rd, offset(rs1)	I	LOAD 0000011	001	–	rd = sext(mem16[addr]), 16-bit value sign-extended	lh x5, 2(x6)	0x00231283 **2298499**
lw	rd, offset(rs1)	I	LOAD 0000011	010	–	rd = mem32[addr]	lw a0, -4(sp)	0xFFC12503 **-4119293**
lbu	rd, offset(rs1)	I	LOAD 0000011	100	–	rd = zext(mem8[addr]), 8-bit value zero-extended	lbu x5, 1(x6)	0x00134283 **1262211**
lhu	rd, offset(rs1)	I	LOAD 0000011	101	–	rd = zext(mem16[addr]), 16-bit value zero-extended	lhu x5, 6(x6)	0x00635283 **6509187**
sb	rs2, offset(rs1)	S	STORE 0100011	000	–	mem8[addr] = rs2[7:0]	sb x5, 3(x6)	0x005301A3 **5439907**
sh	rs2, offset(rs1)	S	STORE 0100011	001	–	mem16[addr] = rs2[15:0]	sh x5, 2(x6)	0x00531123 **5443875**
sw	rs2, offset(rs1)	S	STORE 0100011	010	–	mem32[addr] = rs2	sw x5, 2047(x6)	0x7E532FA3 **2119380899**

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
RV32I: memory ordering and environment								
fence	pred, succ	I (fence)	MISC-MEM 0001111	000	–	Order device I/O and memory accesses: no other hart or device sees an operation in the successor set before any in the predecessor set. Sets are letters from i, o, r, w.	fence rw, w	0x0310000F **51380239**
fence.tso	–	I (fence)	MISC-MEM 0001111	000	fm 1000 pred 0011 succ 0011	FENCE with fm=1000, pred=succ=rw: loads ordered before all later accesses, stores before later stores (total store ordering).	fence.tso	0x8330000F **-2094006257**
pause	–	I (fence)	MISC-MEM 0001111	000	fm 0000 pred 0001 succ 0000	Zihintpause hint: temporarily reduce the instruction retirement rate (e.g. in spin-wait loops). Encoded as FENCE with pred=w, succ=0.	pause	0x0100000F **16777231**
ecall	–	I (system)	SYSTEM 1110011	000	000000000000	Environment call: service request to the execution environment (trap).	ecall	0x00000073 **115**
ebreak	–	I (system)	SYSTEM 1110011	000	000000000001	Breakpoint: return control to a debugging environment (trap).	ebreak	0x00100073 **1048691**

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
M: multiplication and division								
mul	rd, rs1, rs2	R	OP 0110011	000	0000001	rd = (rs1 × rs2)[31:0], lower 32 bits of the product	mul x1, x2, x3	0x023100B3 **36765875**
mulh	rd, rs1, rs2	R	OP 0110011	001	0000001	rd = (rs1 × rs2)[63:32], signed × signed	mulh x1, x2, x3	0x023110B3 **36769971**
mulhsu	rd, rs1, rs2	R	OP 0110011	010	0000001	rd = (rs1 × rs2)[63:32], signed rs1 × unsigned rs2	mulhsu x1, x2, x3	0x023120B3 **36774067**
mulhu	rd, rs1, rs2	R	OP 0110011	011	0000001	rd = (rs1 × rs2)[63:32], unsigned × unsigned	mulhu x1, x2, x3	0x023130B3 **36778163**
div	rd, rs1, rs2	R	OP 0110011	100	0000001	rd = rs1 / rs2, signed, rounds towards zero. x/0 = −1; -2^{31} / −1 = -2^{31}.	div x1, x2, x3	0x023140B3 **36782259**
divu	rd, rs1, rs2	R	OP 0110011	101	0000001	rd = rs1 / rs2, unsigned. x/0 = $2^{32} - 1$ (all ones).	divu x1, x2, x3	0x023150B3 **36786355**
rem	rd, rs1, rs2	R	OP 0110011	110	0000001	rd = rs1 % rs2, signed; a nonzero result has the sign of the dividend. x % 0 = x; -2^{31} % −1 = 0.	rem x1, x2, x3	0x023160B3 **36790451**
remu	rd, rs1, rs2	R	OP 0110011	111	0000001	rd = rs1 % rs2, unsigned. x % 0 = x.	remu x31, x30, x29	0x03DF7FB3 **64978867**

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
B / Zba: address generation								
sh1add	rd, rs1, rs2	R	OP 0110011	010	0010000	rd = rs2 + (rs1 << 1)	sh1add a0, a1, a2	0x20C5A533 **549823795**
sh2add	rd, rs1, rs2	R	OP 0110011	100	0010000	rd = rs2 + (rs1 << 2)	sh2add a0, a1, a2	0x20C5C533 **549831987**
sh3add	rd, rs1, rs2	R	OP 0110011	110	0010000	rd = rs2 + (rs1 << 3)	sh3add a0, a1, a2	0x20C5E533 **549840179**

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
B / Zbb: basic bit manipulation								
andn	rd, rs1, rs2	R	OP 0110011	111	0100000	rd = rs1 & ~rs2	andn x1, x2, x3	0x403170B3 **1076981939**
orn	rd, rs1, rs2	R	OP 0110011	110	0100000	rd = rs1 | ~rs2	orn x1, x2, x3	0x403160B3 **1076977843**
xnor	rd, rs1, rs2	R	OP 0110011	100	0100000	rd = ~(rs1 ^ rs2)	xnor x1, x2, x3	0x403140B3 **1076969651**
clz	rd, rs1	I (unary)	OP-IMM 0010011	001	0110000 00000	rd = number of leading zero bits, counted from bit 31 (32 if rs1 = 0)	clz x3, x4	0x60021193 **1610748307**
ctz	rd, rs1	I (unary)	OP-IMM 0010011	001	0110000 00001	rd = number of trailing zero bits, counted from bit 0 (32 if rs1 = 0)	ctz x3, x4	0x60121193 **1611796883**
cpop	rd, rs1	I (unary)	OP-IMM 0010011	001	0110000 00010	rd = number of 1 bits in rs1 (population count)	cpop x1, x2	0x60211093 **1612779667**
max	rd, rs1, rs2	R	OP 0110011	110	0000101	rd = larger of rs1, rs2 (signed)	max x1, x2, x3	0x0A3160B3 **171008179**
maxu	rd, rs1, rs2	R	OP 0110011	111	0000101	rd = larger of rs1, rs2 (unsigned)	maxu x1, x2, x3	0x0A3170B3 **171012275**
min	rd, rs1, rs2	R	OP 0110011	100	0000101	rd = smaller of rs1, rs2 (signed)	min x1, x2, x3	0x0A3140B3 **170999987**
minu	rd, rs1, rs2	R	OP 0110011	101	0000101	rd = smaller of rs1, rs2 (unsigned)	minu x1, x2, x3	0x0A3150B3 **171004083**
sext.b	rd, rs1	I (unary)	OP-IMM 0010011	001	0110000 00100	rd = sext(rs1[7:0]): bit 7 copied into bits 31..8	sext.b x1, x2	0x60411093 **1614876819**
sext.h	rd, rs1	I (unary)	OP-IMM 0010011	001	0110000 00101	rd = sext(rs1[15:0]): bit 15 copied into bits 31..16	sext.h x1, x2	0x60511093 **1615925395**
zext.h	rd, rs1	R (unary)	OP 0110011	100	0000100 00000	rd = zext(rs1[15:0]): bits 31..16 cleared	zext.h x1, x2	0x080140B3 **134299827**
rol	rd, rs1, rs2	R	OP 0110011	001	0110000	rd = rs1 rotated left by rs2[4:0]	rol x1, x2, x3	0x603110B3 **1613828275**
ror	rd, rs1, rs2	R	OP 0110011	101	0110000	rd = rs1 rotated right by rs2[4:0]	ror x1, x2, x3	0x603150B3 **1613844659**
rori	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	101	0110000	rd = rs1 rotated right by shamt (0..31)	rori x1, x2, 7	0x60715093 **1618038931**
orc.b	rd, rs1	I (unary)	OP-IMM 0010011	101	001010000111	OR-combine per byte: each byte of rd is 0x00 if that byte of rs1 is 0, else 0xFF	orc.b x1, x2	0x28715093 **678514835**
rev8	rd, rs1	I (unary)	OP-IMM 0010011	101	011010011000	rd = rs1 with the byte order reversed	rev8 x1, x2	0x69815093 **1770082451**

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
B / Zbs: single-bit instructions (bit index = rs2[4:0] or shamt 0..31)								
bclr	rd, rs1, rs2	R	OP 0110011	001	0100100	rd = rs1 & ~(1 << rs2[4:0]) (clear bit)	bclr x1, x2, x3	0x483110B3 **1211175091**
bclri	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	001	0100100	rd = rs1 & ~(1 << shamt)	bclri x1, x2, 5	0x48511093 **1213272211**
bext	rd, rs1, rs2	R	OP 0110011	101	0100100	rd = (rs1 >> rs2[4:0]) & 1 (extract bit)	bext x1, x2, x3	0x483150B3 **1211191475**
bexti	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	101	0100100	rd = (rs1 >> shamt) & 1	bexti x1, x2, 5	0x48515093 **1213288595**
binv	rd, rs1, rs2	R	OP 0110011	001	0110100	rd = rs1 ^ (1 << rs2[4:0]) (invert bit)	binv x1, x2, x3	0x683110B3 **1748046003**
binvi	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	001	0110100	rd = rs1 ^ (1 << shamt)	binvi x1, x2, 5	0x68511093 **1750143123**
bset	rd, rs1, rs2	R	OP 0110011	001	0010100	rd = rs1 | (1 << rs2[4:0]) (set bit)	bset x1, x2, x3	0x283110B3 **674304179**
bseti	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	001	0010100	rd = rs1 | (1 << shamt)	bseti x1, x2, 31	0x29F11093 **703664275**

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
Zicsr: CSR access, atomic read-modify-write of one 12-bit CSR (csr = number or name, see CSR table)								
csrrw	rd, csr, rs1	I (csr)	SYSTEM 1110011	001	–	t = CSR; CSR = rs1; rd = t. If rd = x0 the CSR is not read.	csrrw x1, mstatus, x2	0x300110F3 **805376243**
csrrs	rd, csr, rs1	I (csr)	SYSTEM 1110011	010	–	t = CSR; CSR = t | rs1 (set bits); rd = t. If rs1 = x0 the CSR is not written.	csrrs a0, mstatus, t0	0x3002A573 **805479795**
csrrc	rd, csr, rs1	I (csr)	SYSTEM 1110011	011	–	t = CSR; CSR = t & ~rs1 (clear bits); rd = t. If rs1 = x0 the CSR is not written.	csrrc a0, mie, t1	0x30433573 **809710963**
csrrwi	rd, csr, uimm	I (csr, uimm)	SYSTEM 1110011	101	–	like csrrw with zext(uimm[4:0]) instead of rs1	csrrwi a0, mscratch, 5	0x3402D573 **872600947**
csrrsi	rd, csr, uimm	I (csr, uimm)	SYSTEM 1110011	110	–	like csrrs with zext(uimm); uimm = 0 means no write	csrrsi a0, mstatus, 8	0x30046573 **805594483**
csrrci	rd, csr, uimm	I (csr, uimm)	SYSTEM 1110011	111	–	like csrrc with zext(uimm); uimm = 0 means no write	csrrci a0, mstatus, 8	0x30047573 **805598579**

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
Privileged (riscv-privileged.pdf: encodings Ch. 19 Fig. 125, semantics Sec. 3.1.6.1, 3.3, 12.2)								
mret	–	R (priv)	SYSTEM 1110011	000	0011000 00010	Return from a machine-mode trap: pc = mepc; privilege = mstatus.MPP; MIE = MPIE; MPIE = 1; MPP = least-privileged supported mode (U). If the new mode ≠ M: MPRV = 0.	mret	0x30200073 **807403635**
wfi	–	R (priv)	SYSTEM 1110011	000	0001000 00101	Wait for interrupt: may stall until an interrupt is pending. Implementing it as a NOP is legal.	wfi	0x10500073 **273678451**
sret	–	R (priv)	SYSTEM 1110011	000	0001000 00010	Return from a supervisor-mode trap: pc = sepc; privilege = sstatus.SPP; SIE = SPIE. Only with S-mode.	sret	0x10200073 **270532723**
sfence.vma	rs1, rs2	R (sfence)	SYSTEM 1110011	000	0001001	Fence for page-table updates: flush cached translations for virtual address rs1 / address space rs2 (x0 = all). Only with S-mode.	sfence.vma x0, x0	0x12000073 **301990003**

Pseudoinstructions (only those defined in the spec)

Pseudo	Assembled as	Meaning	Example	Example assembled as	Hex · signed int32
nop	addi x0, x0, 0	no operation	nop	addi x0, x0, 0	0x00000013 · 19
mv rd, rs	addi rd, rs, 0	copy register	mv a0, a1	addi x10, x11, 0	0x00058513 · 361747
not rd, rs	xori rd, rs, -1	bitwise NOT	not a0, a1	xori x10, x11, -1	0xFFF5C513 · -670445
seqz rd, rs	sltiu rd, rs, 1	rd = (rs == 0) ? 1 : 0	seqz a0, a1	sltiu x10, x11, 1	0x0015B513 · 1422611
snez rd, rs	sltu rd, x0, rs	rd = (rs ≠ 0) ? 1 : 0	snez a0, a1	sltu x10, x0, x11	0x00B03533 · 11547955
j offset	jal x0, offset	jump, no link	j -16	jal x0, -16	0xFF1FF06F · -14684049
jr rs	jalr x0, 0(rs)	jump to register, no link	jr t0	jalr x0, 0(x5)	0x00028067 · 163943
ret	jalr x0, 0(x1)	return from subroutine (to address in ra)	ret	jalr x0, 0(x1)	0x00008067 · 32871
bgt rs, rt, offset	blt rt, rs, offset	branch if rs > rt (signed): operands swapped	bgt a0, a1, 8	blt x11, x10, 8	0x00A5C463 · 10863715
bgtu rs, rt, offset	bltu rt, rs, offset	branch if rs > rt (unsigned)	bgtu a0, a1, 8	bltu x11, x10, 8	0x00A5E463 · 10871907
ble rs, rt, offset	bge rt, rs, offset	branch if rs ≤ rt (signed)	ble a0, a1, 8	bge x11, x10, 8	0x00A5D463 · 10867811
bleu rs, rt, offset	bgeu rt, rs, offset	branch if rs ≤ rt (unsigned)	bleu a0, a1, 8	bgeu x11, x10, 8	0x00A5F463 · 10876003
csrr rd, csr	csrrs rd, csr, x0	read CSR	csrr a0, mepc	csrrs x10, mepc, x0	0x34102573 · 873473395
csrw csr, rs	csrrw x0, csr, rs	write CSR	csrw mtvec, a0	csrrw x0, mtvec, x10	0x30551073 · 810881139
csrs csr, rs	csrrs x0, csr, rs	set bits in CSR	csrs mstatus, t0	csrrs x0, mstatus, x5	0x3002A073 · 805478515
csrc csr, rs	csrrc x0, csr, rs	clear bits in CSR	csrc mie, t1	csrrc x0, mie, x6	0x30433073 · 809709683
csrwi csr, uimm	csrrwi x0, csr, uimm	write CSR (immediate)	csrwi mscratch, 0	csrrwi x0, mscratch, 0	0x34005073 · 872435827
csrsi csr, uimm	csrrsi x0, csr, uimm	set bits (immediate)	csrsi mstatus, 8	csrrsi x0, mstatus, 8	0x30046073 · 805593203
csrci csr, uimm	csrrci x0, csr, uimm	clear bits (immediate)	csrci mstatus, 8	csrrci x0, mstatus, 8	0x30047073 · 805597299
rdcycle rd	csrrs rd, cycle, x0	read cycle counter, low 32 bits (Sec. 7.1)	rdcycle a0	csrrs x10, cycle, x0	0xC0002573 · -1073732237
rdcycleh rd	csrrs rd, cycleh, x0	cycle counter, high 32 bits	rdcycleh a1	csrrs x11, cycleh, x0	0xC80025F3 · -939514381
rdtime rd	csrrs rd, time, x0	real-time clock, low 32 bits	rdtime a0	csrrs x10, time, x0	0xC0102573 · -1072683661
rdtimeh rd	csrrs rd, timeh, x0	real-time clock, high 32 bits	rdtimeh a1	csrrs x11, timeh, x0	0xC81025F3 · -938465805
rdinstret rd	csrrs rd, instret, x0	instructions retired, low 32 bits	rdinstret a0	csrrs x10, instret, x0	0xC0202573 · -1071635085
rdinstreth rd	csrrs rd, instreth, x0	instructions retired, high 32 bits	rdinstreth a1	csrrs x11, instreth, x0	0xC82025F3 · -937417229

Operand short forms accepted by the tools

Short form	Same as	Meaning	Example	Example assembled as	Hex · signed int32
jal offset	jal x1, offset	call: link register ra	jal 256	jal x1, 256	0x100000EF · 268435695
jalr rs	jalr x1, 0(rs)	call through register	jalr t0	jalr x1, 0(x5)	0x000280E7 · 164071
jalr rd, rs1, offset	jalr rd, offset(rs1)	alternative operand order	jalr x1, x5, 12	jalr x1, 12(x5)	0x00C280E7 · 12746983
lw rd, (rs1)	lw rd, 0(rs1)	offset may be omitted (all loads/stores)	lw a0, (sp)	lw x10, 0(x2)	0x00012503 · 75011
fence	fence iorw, iorw	full fence	fence	fence iorw, iorw	0x0FF0000F · 267386895